

Software Requirements Specification (SRS) Template

Project: Voice Assistant for MultiMedia Controls

Version: 1.0

Authors: C K Gagan Gowda (PES2UG24CS124),
Anudeepak Anpuaraja (PES2UG24CS905),
Nishkaa V (PES2UG24CS907),
Ishanika Vinuthan (PES2UG24CS911)

Date: 3-08-2026

Status: Model Answer / Approved

Revision history

Version	Date	Author	Change summary	Approval
1.0	3-08-2026	Instructor	SRS with diagrams embedded	

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for a Voice Assistant for MultiMedia Controls. It defines functional and non-functional requirements, interfaces, and verification criteria intended for instructors and students to use as a reference.

1.2 Scope

Covers user interactions with the voice assistant for controlling multimedia devices and applications, including voice-based playback, volume, track, and media navigation commands. Covers integration with connected media players and devices, voice command processing, and system-level control functions. Excludes the internal implementation of third-party media players and external services beyond the interfaces required for integration

1.3 Audience

Software Developers, QA Engineers, System Integrators, Maintenance Technicians, and Assessment Evaluators.

1.4 Definitions

List of acronyms:

MQTT: Message Queuing Telemetry Transport

UART: Universal Asynchronous Receiver Transmitter

2. Overall description

2.1 Product perspective

The system is a standalone, hardware-based multimodal multimedia controller. It acts as a bridge device, capturing hands-free user inputs both audio and visual and translating them into standardized media commands to control a connected audio system or display.

2.2 Major product functions & Deliverables

- **Host Runtime Environment:** Configured execution setup running on the laptop (Python 3.x runtime) connected to onboard or USB webcams and microphones.
- **Voice Processing Engine:** Speech recognition pipeline that continuously captures audio buffers and converts keyword triggers into discrete media commands (Play, Pause, Resume, Next, Previous, Volume Up, Volume Down).

- **Media Command Interface:** System-level bindings that translate the parsed intents from the ML engines into actual OS media controls.
- **Project Documentation:** Complete hardware schematics (pinouts), a configured source code repository with requirements.txt, and deployment instructions.

2.3 User roles and characteristics

- **End-User:** Requires intuitive, hands-free interaction. Expects low-latency execution of media commands and high accuracy, regardless of ambient room noise or varying lighting conditions.
- **System Maintainer/Developer:** Requires clear architectural separation between the machine learning pipelines and the hardware actuation layer, alongside accessible debug logs for troubleshooting false-positive triggers.

2.4 Operating environment

- **Host OS:** Desktop environment (Windows 10/11, macOS, or Ubuntu Linux) running Python 3.8+.
- **Hardware Peripherals:** Integrated or external USB webcam, microphone array, and USB/Bluetooth microcontrollers (ESP32/Raspberry Pi).

2.5 Constraints

- **Peripheral Access Lock:** The system requires persistent access to the webcam and microphone, which can create resource conflicts if other software (e.g., video conferencing applications) locks the video input stream.
- **Lighting and Acoustic Dependence:** Visual gesture detection depends on the camera's field of view and room illumination, while voice commands require filtering to isolate user commands from ambient laptop speaker output.
- **Multi-User & Visual Ambiguity** If multiple people enter the camera's field of view, or if background objects resemble hand landmarks, MediaPipe may track the wrong hand or alternate rapidly between users, causing erratic volume or navigation commands.
- **Battery & Thermal Throttling** Continuously capturing raw video frames at high frame rates (30–60 FPS) and executing real-time landmark extraction increases power consumption and heat generation, which can trigger thermal throttling or rapidly drain laptop battery when unplugged.
- **Distance & Spatial Range** Gesture tracking drops drastically in reliability beyond 1.5 to 2 meters from standard webcams due to lower resolution of hand features, while fast movements cause motion blur that destabilizes MediaPipe landmark coordinates.

3. External interface requirements

3.1 User interfaces

- **Primary UI:** Minimalist Head-Up Display (HUD) overlay on the laptop screen showing live webcam feed, real-time MediaPipe hand landmark tracking skeletons, recognized voice commands, and volume/media state changes.
- **Non-Visual Feedback:** Audio cues (distinct operational beeps/chimes) confirming successful command execution for eyes-free control.
- **Accessibility:** High-contrast gesture boundary markers, adjustable visual HUD scale, customizable gesture/voice sensitivity thresholds, and spoken audio confirmation toggles for visually impaired users.

3.2 Hardware interfaces

- **Camera Sensor:** Integrated laptop webcam or external USB/CSI camera module capturing real-time video feed (minimum 720p @ 30 FPS).
- **Microphone Array:** Integrated laptop dual-mic array or USB/I2S microphone module for continuous audio stream capture.
- **Microcontroller Node (Optional Hybrid Setup):** ESP32 or Raspberry Pi connected via USB/UART featuring physical status LEDs, physical mute switches, or secondary IR transmitters.
- **Host Audio System:** Onboard laptop speakers or connected Bluetooth/3.5mm audio endpoints for media output and audio feedback.

3.3 Software interfaces

- **Media Pipe ML SDK:** Python framework for real-time hand-landmark detection and multi-point coordinate extraction.
- **Speech Recognition Engine:** Local speech parsing library (Vosk, SpeechRecognition, or Whisper) processing PyAudio stream buffers.
- **OS Media Event Hooks:** Native system APIs and keystroke simulation libraries (py autogui, pynput) to send global play, pause, volume, and track navigation events.
- **Serial Protocol Drivers:** pySerial interface handling incoming hardware interrupts or telemetry from an attached ESP32 or Raspberry Pi.

3.4 Communications

- **Host Inter-Process Communication (IPC):** Thread-safe python queues and events bridging concurrent webcam capturing, audio processing, and system execution threads.
- **Microcontroller-to-Host Link:** USB-Serial (UART at 115200 baud) or local Wi-Fi protocol (Lightweight MQTT / WebSockets) for hardware trigger transmission.

- **Data Security & Privacy:** 100% offline local processing with no continuous audio or video streaming sent to external cloud APIs, preventing unauthorized network interception.

4. System features (detailed)

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
FR-01	The system shall accept voice commands from the user through a connected microphone.	FR	High	User	The system correctly captures a spoken command and passes it for processing.	Requires microphone connected to the Pi/ESP.
FR-02	The system shall recognize commands for playing media.	FR	High	User	When the user gives a valid play command, the system starts media playback.	Depends on speech recognition and media player integration.
FR-03	The system shall recognize commands for pausing and resuming media playback.	FR	High	User	Pause and resume commands	Requires media player control interface.

					produce the corresponding playback actions.	
FR-04	The system shall pause the currently playing media when the user issues a pause command.	FR	High	User	Say "pause" during playback and verify that playback stops at the current position.	Requires media player control interface.
FR-05	The system shall increase media volume when the user issues a volume-up command.	FR	High	User	Say "volume up" and verify that the media volume increases by the configured increment.	Depends on system/media-player volume control.
FR-06	The system shall decrease media volume when the user issues a volume-down command.	FR	High	User	Say "volume down" and verify that the media volume decreases by the configured increment.	Depends on system/media-player volume control.
FR-07	The system shall mute or unmute media audio when the corresponding voice command is issued.	FR	Medium	User	Issue "mute" and "unmute" commands and verify the audio state changes accordingly.	Requires operating-system or media-player audio control.
FR-08	The system shall play the next media item when the user issues a	FR	High	User	During playback, say "next" and verify that the following media item is selected and played.	Depends on media-player playlist functionality.

	next-track command.					
FR-09	The system shall play the previous media item when the user issues a previous-track command.	FR	High	User	Say "back" and verify that the previous media item is selected or replayed according to the media-player behavior.	Depends on media-player playlist functionality.
FR-10	The system shall provide feedback after processing a voice command.	FR	Medium	User	Issue a valid command and verify that the system provides an appropriate audio, visual, or textual indication.	Feedback mechanism depends on available hardware/software.
FR-11	The system shall notify the user when a voice command is not recognized.	FR	Medium	User	Issue an unsupported or unclear command and verify that the system indicates that the command was not understood.	Requires command recognition and feedback modules.
FR-12	The system shall support a predefined set of voice commands for all implemented multimedia operations.	FR	High	Developer/ User	Test every command in the supported command set and verify that each triggers its corresponding operation.	Command vocabulary must be defined during implementation.
FR-13		FR	High	Developer		Depends on Pi/ESP and media-pla

	The system shall execute recognized multimedia commands on the connected media player.				Issue each supported command and verify that the corresponding action occurs on the target media player.	voice assistant interface.
FR-14	The system shall allow the user to control multimedia playback without directly interacting with the media player interface.	FR	Medium	User	Perform playback, volume, and track operations using only supported voice commands and verify successful control.	Voice assistant acts as the primary control interface.
FR-15	The system shall operate on the selected hardware platform and communicate with the media player to perform supported controls.	FR	High	Developer	Deploy the system on the selected Pi/ESP-based setup and verify that voice commands control the connected media player.	Depends on hardware configuration, Python software, and MediaPipe/media-player integration.

5. Non-functional requirements (detailed)

NFRs below are measurable and tied to test plans.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
NF-1	End-to-end latency from spoken command / gesture to executed media action shall be ≤ 1.5 seconds under normal conditions.	Performance	High	90th percentile latency ≤ 1.5 s measured across 50 test commands. Test: TC-Perf-01
NF-2	The system shall correctly recognize supported voice commands with $\geq 90\%$ accuracy in typical indoor ambient noise (≤ 60 dB).	Reliability	High	Command recognition accuracy $\geq 90\%$ across a 100-command test set. Test: TC-Rel-01
NF-3	All audio and video processing shall occur 100% locally on-device; no raw audio/video shall be transmitted to external/cloud services.	Security/Privacy	High	Network traffic capture during operation shows zero outbound audio/video streams. Test: TC-Sec-01

NF-4	The system shall provide accessible feedback (audio cues and/or spoken confirmation) for every executed command to support visually impaired users.	Usability/Accessibility	Medium	Manual accessibility walkthrough confirms audio confirmation for all FR-02–FR-09 commands. Test: TC-UX-01
NF-5	The host software shall run on Windows 10/11, macOS, and Ubuntu Linux with Python 3.8+, without code changes.	Portability/Compatibility	Medium	Successful install and command execution verified on all three target OSes. Test: TC-Port-01
NF-6	The system shall log recognized commands, timestamps, and execution status to a local debug log for troubleshooting false-positive triggers.	Maintainability	Medium	Log file inspection confirms timestamped entries for each test command, without storing raw audio/video. Test: TC-Maint-01

5.1 Security

5.1.1 Security Objectives

1. Prevent unauthorized or accidental activation of multimedia controls from ambient noise, background conversation, or false gesture detection.

2. Ensure no audio or video captured by the microphone/webcam is stored or transmitted beyond what is required for real-time local command processing.
3. Protect the microcontroller-to-host communication link (USB/Wi-Fi) from unauthorized access or command injection.
4. Ensure the system fails safe — an unrecognized or ambiguous input results in no action rather than an unintended media command.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
PRJ-SR-001	The system shall process all audio and video locally, with no raw stream transmitted to external servers.	Security	High	TC-Sec-01
PRJ-SR-002	The system shall require a distinct wake word or trigger gesture before executing any multimedia command.	Security	High	TC-Sec-02
PRJ-SR-003	Communication between the microcontroller (ESP32/Pi) and host over Wi-Fi (MQTT/WebSockets) shall use an encrypted channel (minimum WPA2/TLS).	Security	Medium	TC-Sec-03
PRJ-SR-004	Debug logs shall record command and execution metadata only, never raw audio/video content.	Security	Medium	TC-Sec-04
PRJ-SR-005	The HUD/debug interface exposing the live camera feed shall be accessible only on the local device, not exposed over the network.	Security	High	TC-Sec-05

6. Quality attributes & Acceptance tests

- **Exit criteria for acceptance:** All high-priority functional requirements implemented and verified, no critical NFR failures, and RTM shows all test cases passed.
- **Acceptance test suites:** Voice Command Recognition, Playback Control (Play/Pause/Resume), Volume Control (Up/Down/Mute), Track Navigation (Next/Previous), Latency/Performance, Security & Privacy, and Accessibility tests.

7. System models and diagrams

UML Use-Case diagram

The Voice-Controlled Media Player System provides an interface for controlling media playback through spoken commands. The user interacts with the system through a microphone, while the Raspberry Pi processes the captured voice command and controls the media playback device. The primary use cases include capturing and recognizing voice commands, validating commands, controlling playback, adjusting volume, and providing audio output. A secondary set of use cases addresses device initialization, hardware connectivity, invalid command handling, feedback, and recovery from input or playback errors.

Actors: User, Device Operator, Speaker / Audio Output.

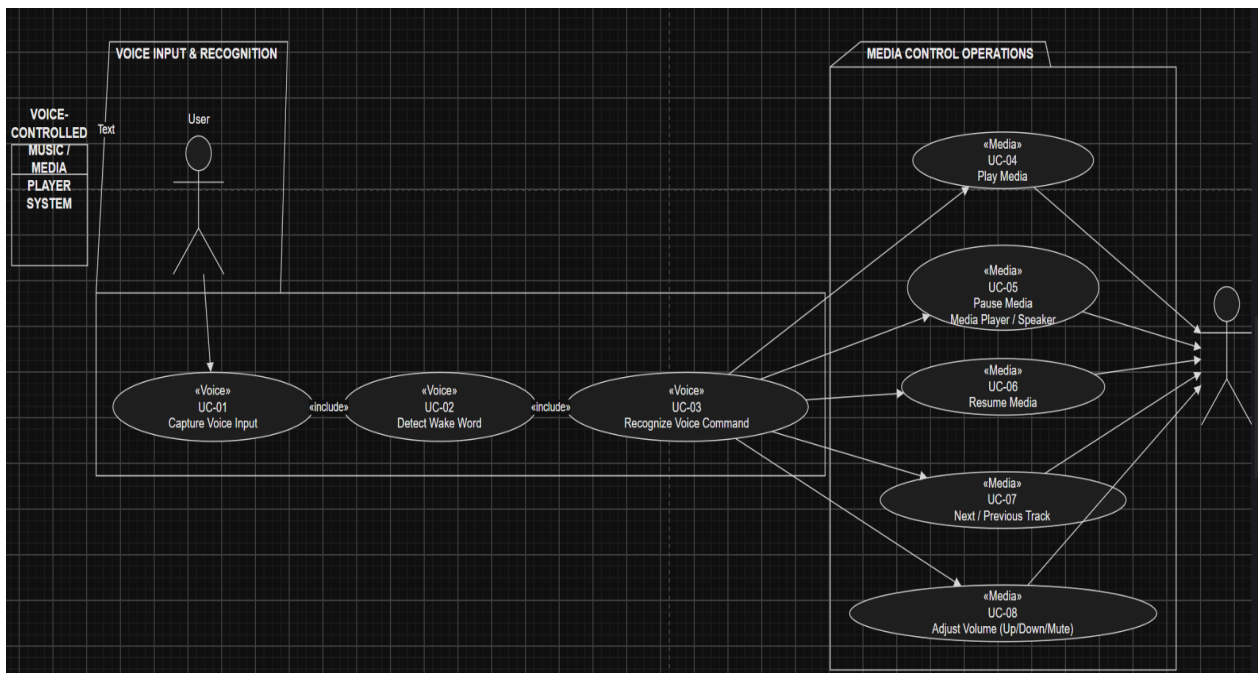


Diagram 7.1 Voice Command Pipeline Flow

User → Capture Voice Input (Mic) → Detect Wake Word → Recognize Voice Command → Execute Media Action (Play/Pause/Volume/Skip) → Media Player / Speaker

The user initiates the interaction by speaking, which is captured (UC-01) and strictly gated by a wake word (UC-02) before processing, satisfying PRJ-SR-002.

Recognized commands (UC-03) branch into specific media operations (UC-04 to UC-08).

All media control use cases interact directly with the external Media Player/Speaker actor to alter physical playback states.

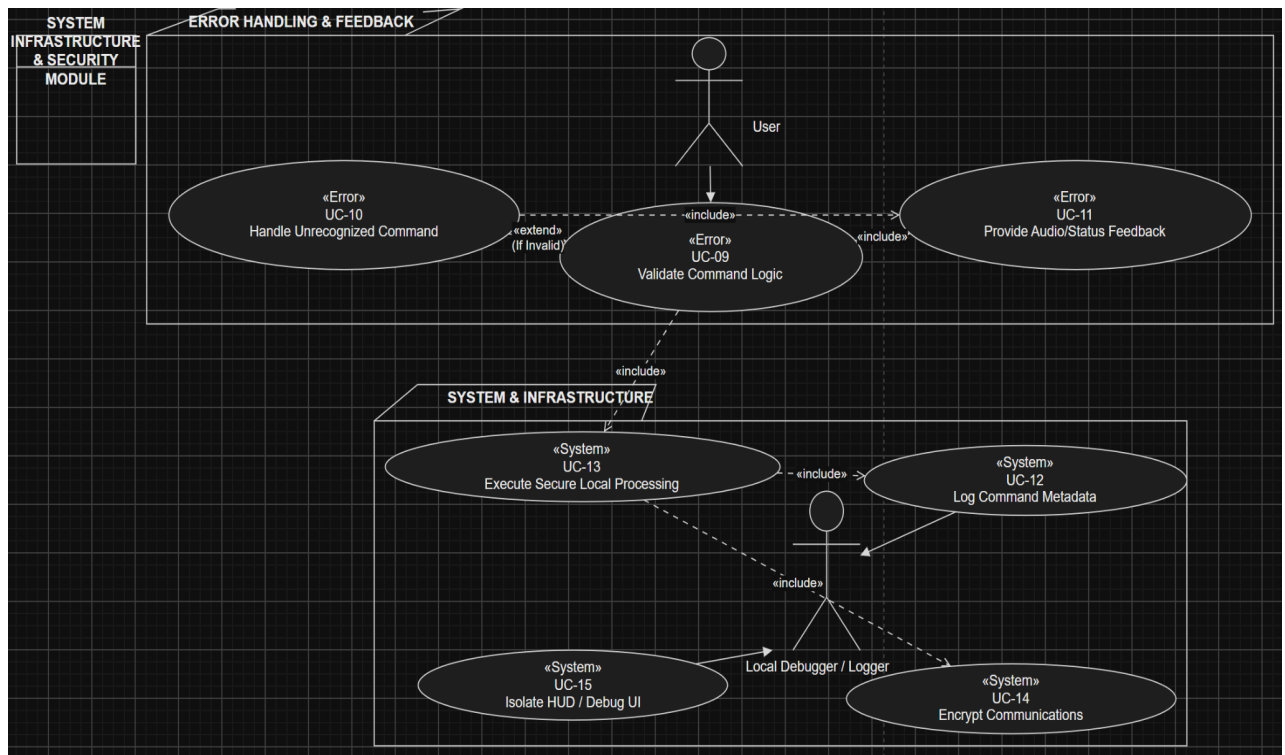


Diagram 7.2: Infrastructure & Security Flow

This represents the background processes, security checks, and error handling that occur seamlessly behind the scenes.

Startup: Device Power On → Initialize Hardware (UC-16) (Mic, Speaker, OS modules) → System Ready

User Input → Validate Command Logic → [If Invalid: Handle Unrecognized Command] → Execute Secure Local Processing → Encrypt Communications / Log Metadata / Provide Status Feedback → User / Local Debugger

Validation (UC-09) triggers error handling (UC-10) only if a command fails to parse, utilizing an <<extend>> relationship.

Both successful validations and unrecognized commands mandate user feedback (UC-11) per FR-14.

The system enforces 100% local processing (UC-13) and strips raw audio from logs (UC-12), adhering strictly to privacy requirements PRJ-SR-001 and PRJ-SR-004.

8. Requirements Traceability Matrix (RTM)

The following Requirements Traceability Matrix maps each functional, non-functional, and security requirement to its corresponding design specification, implementation module, and verification test case. The RTM ensures that every requirement defined in Sections 4 and 5 is traceable through design and implementation to system validation.

Status codes: N = Not Implemented / Not Verified, P = Partially Implemented, A = Accepted / Verified.

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
FR-01	System shall capture voice input through a connected microphone	4.1 / DS-Voice-01	Microphone Interface	TC-VOICE-01	N	Requires microphone connection
FR-02	System shall recognize commands for playing media	4.1 / DS-Voice-02	Voice Input Handler	TC-VOICE-02	N	Command initiation
FR-03	System shall recognize pause and resume commands	4.1 / DS-Voice-03	Audio Preprocessor	TC-VOICE-03	N	Noise handling / normalization
FR-04	System shall pause currently playing media on pause command	4.1 / DS-STT-01	Speech Recognition Module	TC-STT-01	N	Speech-to-text stage
FR-05	System shall increase media volume on volume-up command	4.1 / DS-CMD-01	Command Parser	TC-CMD-01	N	Valid command mapping

FR-06	System shall play selected/current media on play command	4.2 / DS-PLAY-01	Media Controller	TC-PLAY-01	N	Playback initiated
FR-07	System shall pause playing media on pause command	4.2 / DS-PLAY-02	Media Controller	TC-PLAY-02	N	Pause operation
FR-08	System shall resume paused media on resume command	4.2 / DS-PLAY-03	Media Controller	TC-PLAY-03	N	Resume operation
FR-09	System shall skip to next track on next command	4.2 / DS-NEXT-01	Playlist Controller	TC-NEXT-01	N	Next-track operation
FR-10	System shall return to previous track on previous command	4.2 / DS-PREV-01	Playlist Controller	TC-PREV-01	N	Previous-track operation
FR-11	System shall increase playback volume on volume-up command	4.2 / DS-VOL-01	Audio Controller	TC-VOL-01	N	Volume increment
FR-12	System shall decrease playback volume on volume-down command	4.2 / DS-VOL-02	Audio Controller	TC-VOL-02	N	Volume decrement
FR-13	System shall mute / unmute audio on mute command	4.2 / DS-MUTE-01	Audio Controller	TC-MUTE-01	N	Mute toggle
FR-14	System shall provide feedback after command is processed	4.3 / DS-FB-01	Feedback Handler	TC-FB-01	N	Audio/status feedback
FR-15	System shall handle unrecognised commands without crashing	4.3 / DS-ERR-01	Error Handler	TC-ERR-01	N	Invalid command handling
NF-01	End-to-end latency $\leq$ 1.5 s (90th percentile, 50 commands)	5 / DS-PERF-01	Voice + Command Pipeline	TC-PERF-01	N	Response-time measurement
NF-02	Voice command recognition accuracy $\geq$ 90 % in indoor noise $\leq$ 60 dB	5 / DS-ACC-01	Speech Recognition Module	TC-ACC-01	N	Accuracy test with command set
NF-03	All audio/video processing shall occur	5 / DS-SEC-01	All processing	TC-SEC-01	N	Network capture shows zero outbound

	100 % locally; no cloud streams		modules			
NF-04	Accessible audio/spoken feedback for every executed command	5 / DS-UX-01	Feedback Handler	TC-UX-01	N	Accessibility walkthrough – FR-02–FR-09
NF-05	Runs on Windows 10/11, macOS, Ubuntu Linux with Python 3.8+	5 / DS-PORT-01	Initialization Module	TC-PORT-01	N	Verified on all 3 OSes
NF-06	Timestamped debug log for every command; no raw audio/video stored	5 / DS-MAINT-01	Logger Module	TC-MAINT-01	N	Log file inspection
PRJ-SR-001	System shall process all audio/video locally; no raw stream to external servers	5.1 / DS-SEC-01	All processing modules	TC-SEC-01	N	Privacy requirement
PRJ-SR-002	System shall require a distinct wake word before executing any command	5.1 / DS-SEC-02	Command Validator	TC-SEC-02	N	Wake-word gate
PRJ-SR-003	Wi-Fi/MQTT/WebSocket link to microcontroller shall use WPA2/TLS encryption	5.1 / DS-SEC-03	Comm s Layer	TC-SEC-03	N	Encrypted channel check
PRJ-SR-004	Debug logs shall store command metadata only; never raw audio/video	5.1 / DS-SEC-04	Logger Module	TC-SEC-04	N	Log content audit
PR-SR-005	HUD/debug interface shall be accessible on local device only; not network-exposed	5.1 / DS-SEC-05	HUD / Debug UI	TC-SEC-05	N	Network scan confirms no exposure